

Tethered Bicopter Motor Stabilization Study

Caitlin Bouris
Mechanical Engineering
University of Illinois Chicago
Chicago, IL
cbour3@uic.edu

Ever Gonzalez
Mechanical Engineering
University of Illinois Chicago
Chicago, IL
egonz44@uic.edu

Tommy Borowiec
Mechanical Engineering
University of Illinois Chicago
Chicago, IL
tpb5@uic.edu

Kate Holian
Mechanical Engineering
University of Illinois Chicago
Chicago, IL
kholia2@uic.edu

Campbell Crafts
Mechanical Engineering
University of Illinois Chicago
Chicago, IL
ccrafts2@uic.edu

I. ABSTRACT

This report presents the design and fabrication of a mechanically tethered two-dimensional drone constrained to motion within a vertical plane. The objective was to develop a low-cost, programmable platform capable of autonomous stabilization for use in robotics instruction and laboratory research. Design results show that a four-bar linkage attached to a gantry cart & rail system served as the most effective tether. The chassis was 3D printed with PLA plastic and attached to the linkage. The electronics consist of an Arduino Uno as the controller, two brushless motors attached to electronic speed controllers (ESCs), and an Inertial Measurement Unit (IMU). Power from a 4S LiPo battery is allocated to the electronics through a power distribution board (PDB). Proportional - derivative (PD) controls were implemented in the Arduino Uno using the IMU's angular position and acceleration readings as inputs. The PD controller adjusts the throttle sent to each motor, allowing the drone to self-stabilize when disturbed. Proportional gains were determined through iterative tuning based on IMU data and visual observation. This system supports safe indoor operation in accordance with FAA and OSHA guidelines and provides a reliable platform for demonstrating planar drone dynamics. The complete system provides a solid foundation for control tuning, validation of robotic theory, and engaging demonstrations for engineering students.

II. INTRODUCTION

In the robotics field, simulation is a development technique that conserves resources during the creation of devices [1]. Designers can work on new or complex mechanisms before integrating them into complete devices. Additionally, simulations allow for repeated and rigorous testing without damaging a full product. Drones, for example, can carry expensive sensors for various types of data collection. It is economical to establish confidence in a drone's intended motion prior to equipping its sensors. One underdeveloped mechanism in drone motion is physical plane restriction. This is a method of motion control that reduces a drone's domain to a plane or axis using a tether. Tethering a drone can remedy the uncertainties of its alternative: using free moving drones in cases when only planar or axial motion is desired.

The project's sponsor, Professor Pranav A. Bhounsule, requires a working prototype of a drone physically constrained to a vertical plane. The drone must have 3 degrees of freedom: y-translation, z translation, and rotation about the x-axis (roll). It must self-stabilize about the roll axis. The primary purpose of the drone is for demonstrations that can be used for class assignments, as well as simulating 2D motion theory in the sponsor's robotics laboratory.

Elements of the project's necessary domain restrictions exist throughout the robotics field. The software for 2D motion is occasionally embedded in drones with a 3D domain. It can be more efficient for

drones with a target in a 3D space to create paths by combining 2D routes [2]. While 2D pathing is implemented, planar motion is not the output. Plane-restriction is still used in the robotics field. Most often, it occurs in devices restricted to the transverse plane, either on ground [3] or aurally [4]. The benefits of motion about the horizontal plane include a broad range of applications and consistent transverse physics throughout its domain. By contrast, a drone moving about a vertical plane combines horizontal motion and vertical motion. This adds a level of complexity to the motors and their controls.

The dynamic modeling of a bicopter is one way unwanted travel can be produced. A bicopter can be modeled with two separate propulsion units mounted on a chassis. A dual rotor aircraft has six degrees of freedom, but only four input controls, making it an under actuated system [3,5]. The vertical movement is generated by two propellers that can increase or decrease speed simultaneously, leading to pitch and roll motions. The biggest issue comes with stabilization control in a dual motor device. The kinetic model of the drone can be analyzed thoroughly through MATLAB/Simulink models [5]. Dr. Bhounsule has extensive research on this leveling and stabilization model that can be provided in the construction of this drone [3].

This drone will have the ability to fly under 400 feet therefore it must abide by the regulations [6] created by the Federal Aviation Administration. More specifically, the drone will comply with the 2025 Drone Integration Below Visual Line of Sight (BVLOS) Concept of Operations (CONOPS) article [6]. This article's main objective is to provide regulations for drone operations below 400 feet above ground level (AGL) as well as airspace classes for B, C, D, E, and G. The airspace classes B-E are controlled, and G is the only uncontrolled class. In the CONOPS article Part 107 [7] is mentioned which is a law that applies to small Unmanned Aircraft Systems (UAS) weighing less than 55lbs. Under Part 107 you must abide to the following, drone registration, remote pilot certification (or operating under a certificated pilot), keep it in VLOS, and operating limitations (altitude/airspace, yield to manned aircraft, etc.). The drone is being developed for research purposes and will operate in class G airspace below 400 feet AGL.

The drone will be tethered to the table in which it must follow another set of regulations made by the FAA.

The next guideline that the drone must adhere to is the Checklist for Public Safety Use of Actively Tethered UAS [8] because it will be demonstrated in a building with bystanders near the device. According to the Checklist for Public Safety Use of Actively Tethered UAS, it must stay in VLOS, cannot operate over nonparticipating persons, and yield the right of way to other aircraft. It is also mentioned that if the drone is disconnected from the tether, it must initiate a controlled landing.

The drone will be used in a lab for research and educational purposes therefore it must hold to the lab safety standards created by OSHA's guide on Small Unmanned Aircraft Systems (SUAS) [9]. It was made in accordance with the FAA's regulations and public safety in mind. In the guides of Appendix A, it covers step by step the best practices for SUAS preflight and post flight procedures. For example, before the flight, it must be made sure the drone flight is qualified and approved to operate. As for a post flight procedure example includes a curated checklist for the specific drone as in checking the rotor blades for any damage or debris and resetting the drone to be used again.

This report describes the design process of a 2D tethered UAV. The technical content section utilizes a house of quality (HOQ) to understand and connect the design's objectives to its metrics. Decision matrices are then utilized to select design alternatives for certain components of the apparatus. An embodiment design is included at the end.

III. DESIGN

The design process utilizes an HOQ (Figure 1). The HOQ relates the design objectives (left column) to product metrics (top row) in the center grid. It also relates the metrics to other metrics in the top triangle of the chart. The relations are depicted by strength: strong, medium, weak, or none. The priority of each objective is indicated in the second column of the HOQ.

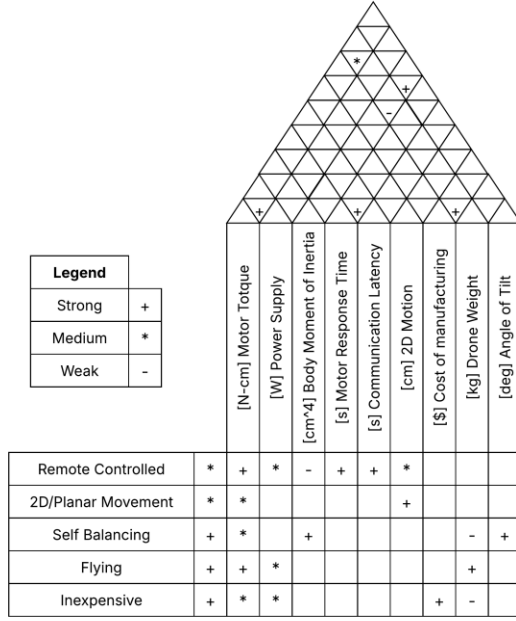


Figure 1: House of Quality for a tethered drone design

Some clarifications are necessary for the HOQ in Figure 1. “Remote Controlled” was assigned medium importance because the user is intended to program the device. Perfecting the controls, therefore, was not considered a goal of the initial device manufacturing. The objective “2D/Planar Movement” was assigned medium importance as exact 2D restriction was deemed difficult to implement in combination with the other objectives. The metric for “2D Motion” is listed as cm. The measurement quantifying this metric is the distance travelled outside of the vertical plane, which would create 3D motion.

Defining the project’s objectives and metrics in the HOQ provided a basis for choosing design alternatives. Specifically, the vertical and horizontal tethers, motors, and microcontroller were chosen using decision matrices (Tables 1-4). The mechanisms were assigned appropriate objectives related to the HOQ. The objectives were given weighted importance, and each design alternative was rated from 1 to 10 (10 being the most favorable) according to its relation to the objectives. The ratings were then multiplied by the objectives’ weights and totaled to assign a score to each alternative.

Initially the mechanism for horizontal, y-direction, motion was addressed. For horizontal

translation, four options were considered: a gantry cart and rail system [10-11], a four-bar linkage, a thrust ball bearing, and a thrust needle bearing were considered. The gantry cart & rail system consists of a horizontal rail with a gantry cart on the top; this results in a one-dimensional translation with low friction. The two bearings support rotational motion in a singular plane, allowing for motion in the horizontal (y) direction but also motion in the forward-backward (x) direction. Similarly, a horizontal four-bar linkage supports y-translation, but also results in some x-translation.

Table 1: Decision matrix for horizontal translation mechanism

Mechanism	Low Cost	2-D Movement	Light Weight	Ease of Use	Rank
Weight	0.3	0.2	0.2	0.3	1
Gantry Cart & Rail	1	10	5	10	6.3
Four-Bar Linkage	10	1	1	5	4.9
Thrust Ball Bearing	7	5	10	1	5.4
Thrust Needle Bearing	3	5	10	1	4.2

A horizontal gantry cart and rail system was chosen. Although the most expensive option, the ease of use from its low friction characteristics and its ability to constrain motion to one dimension made this system the best choice.

The mechanism responsible for the z-axis constraint or “tether” between the gantry and the bicopter was evaluated. Three linkages were considered, along with a second gantry cart and rail system and a straight boom stick. The first linkage discussed was a four-bar linkage, which uses four equal-length bars connected by pivot fasteners to keep the base and output links in a strictly vertical orientation through the linkage’s motion. Purchasing a manufactured camera jib with a four-bar linkage was also considered [12]. Finally, the Peaucellier–Lipkin Linkage is composed of eight specific unequal lengths,

which provides an output point that traces an almost perfectly straight line. By using a vertical rail mounted on a horizontal gantry cart, the system gains a second constrained axis resulting in precise and fully two-dimensional motion. A fifth option was also considered, a PVC boom stick; this includes a PVC pipe connected to a secure base via a spherical joint and a pin joint connecting to the bicopter.

Table 2: Decision matrix for tethering mechanism

Mechanism	Low Cost	2D Motion	Light Weight	Ease of Use	Rank
Weight	0.3	0.2	0.2	0.3	1
Four-Bar Linkage	10	3	8	10	8.2
Gantry Cart & Rail	3	10	10	5	7.0
Boom Stick	5	1	3	8	4.7
Camera Jib	1	5	1	3	2.9
Peaucellier–Lipkin Linkage	8	8	5	1	5.3

The objective “Light Weight” is important to achieve a flying drone. A lighter tether will require less torque and power from the motors, conserving energy and costs. “2D Motion” as an objective describes how well the tether restricts the drone to the vertical plane. To achieve two-dimensional motion, the motion must be constrained to y and z directions. Assuming the designs are attached to the horizontal gantry cart, the evaluation of “2D motion” is dependent on how well the mechanisms for vertical motion constrain the bicopter to movement only in the z direction. The vertical gantry cart and rail system provides perfect constraint, as the cart can only translate up and down along the rail. The linkage mechanisms can guide the drone in the z direction, but their motion follows an arc, meaning that the movement is not fully restricted to a vertical line. The boom stick alone fails to constrain motion to the y-z plane, as it allows spherical motion which includes all three special dimensions. However, if the boom stick is sufficiently long and the operating region is small, the resulting arc produces minor deviation for the y-z plane. Ultimately, to maintain a low-cost design and ease of implementation, a four-bar linkage was determined as the best vertical constraint.

Similarly, the motors for the drone were selected using a decision matrix (Table 3). During a meeting conducted on October 21, 2025, Dr. Bhounsule stated that he has worked on multiple drones and recommended brushless pancake motors to be used. Research and analysis have proven these motors to be the most reliable and well-versed type for the use of drones through experiments and simulations [13]. The motor alternatives within the decision matrix are classified under X2212, 5010, MT2213, F15845, and D4215 [14-18]. The X2212 motor is a moderate middle ground motor. The recommended propeller for this motor is 9.4 inches long. The 5010 motor is the most impactful motor of the three regarding torque. This is mainly due to its high continuous power output, low KV rating, and long recommended propeller length of 14-16 inches. However, the motor is the biggest and heaviest when comparing it to the rest. The MT2213 motor stands greatly due to specification values such as its relatively high current output and low weight. Its moderate KV rating also aligns well with its recommended propeller length and with the objective of the design, best suited for response and thrust. The F15845 motor is the lightest and smallest of the five motors. It provides the least power and has the highest KV rating. The D4215 motor is the heaviest and most expensive in the group. However, it has the lowest KV rating; pairing this along with a long propeller will result in immense torque values. Before taking a deeper look into the role these components will play in the device, it is important to note the significance of a motor and its configuration when it comes to designing a drone [19-20]. These configurations maximize and/or minimize the objective strived for in this project, which will play in the decision for the best motor option.

Table 3: Decision matrix for motors

	Cost	Response	Efficiency	Torque	Thrust	Rank
Weight	0.1	0.3	0.1	0.2	0.3	1
X2212	7	7	6	3	3	4.9
5010	5	7	10	10	5	7.1
MT2213	9	10	8	4	10	8.5
F15845	10	1	1	1	1	1.9
D4215	1	4	9	9	8	6.4

The major criteria seen in the motor decision matrix include response, efficiency, torque, and thrust. These are some key aspects to investigate when determining the best motor for the drone in the sense of this specific project. Each one's basis is clarified as follows. Beginning with response, which refers to how quickly the motor can accelerate, decelerate, and change thrust. Response is a significant characteristic to consider when location precision is key. Ensuring that the drone responds accurately and swiftly to commands is a necessity judging that the project requires precise drone actions. This component allows the drone to complete desired project objectives effectively, mainly maintaining its stability without any fault. Justifying its extensive weight factor.

Efficiency is defined as how well the motor, propeller, and battery are matched for the entire system. When looking at the entire system, the drone is relatively small compared to the boom. So, alongside the drones' own weight, the weight and stiffness of the linkage will also be at play. This balance between thrust, weight, and any reactionary forces within the linkage is the focus when thinking about efficiency. However, efficiency is not a significant consideration in this project, given that the drone is not expected to operate for extended durations or cover long ranges. In other words, if the drone can operate effectively and efficiently through a short period of time, like during a demonstration, it will suffice. This results in the low weight factor.

Torque and thrust are the final concepts to consider. A drone equipped with motors that deliver adequate torque and precise thrust control---sufficient to support both its own weight and the loads imposed by the four-bar linkage---is fundamental to achieving accurate and stable flight. Concentrating on precision and not speed will enable the drone to achieve desired tasks. Not only will it result in high accuracy positioning, but a focus on these two components will allow for stronger drone control for repositioning and auto stabilization. The MT2213 motor has the best overall rank values for each category other than torque. This is no issue however since the prioritized categories include thrust and response, as they have the highest weight factor, and the MT2213 scored the best score for each of those categories due key specifications such as high current, moderate KV

rating, and high thrust-to-weight ratio. This results in the highest-ranking alternative of the 5, making this motor the best option for this project.

Shifting gears to what is going to end up moving the motors mentioned above, a controller that acts as the brain of the entire electronic system involved within the drone will be needed. Typically, drones are controlled through an electronic component called the "Flight Controller". A Flight Controller is the central circuit board of a drone or UAV and is responsible for interpreting user inputs and sensor data to manage flight stability and movements. The issue with this however is that most Flight Controllers are designed and configured for the purpose of operating quadcopters, or drones with four propellers, and the drone used in this project is a bi-copter, a two-propeller drone. Although it is possible to use a Flight Controller on a bi-copter, it poses the issue of having to adjust and configure the two unused propeller connections due to the Flight Controller controlling the drone while still thinking there are four propellers if left unchecked. This can result in misinterpretation and errors which can then result in poor and inaccurate drone movement. Adjusting the configurations requires extensive experience and knowledge in flight controller firmware. In other words, adjusting these configurations can be difficult and tedious, especially starting with no knowledge in that specific field. Alternatively, an open-source, programmable microcontroller board can be used to interpret and manage commands and stability with the help of some additional electronic components. An Arduino board fits this perfectly. Different variations and arrangements with Arduino were considered, these include Arduino Uno R3 (9V), Arduino Uno R4 (9V), Arduino Uno R3 (5V), and Arduino Uno R4 (5V). The "9V" in parentheses indicate that the board is powered with the 12V BEC (Battery Eliminator Circuit) on the PDB (Power Distribution Board) stepdown to 9V with a DC-DC Buck Converter, while the "5V" means that the board is powered with the 5V BEC and simply attached to a voltage input pin. These electronic parts and arrangements will be explained in further detail below. When it came to determining and finalizing a decision on how to effectively control and power the drone apparatus, the criteria shown in Table 4 represent the key pillars to figuring that out. The Arduino variations alongside the Flight Controller

were all taken into consideration during initial research of potential systems for controllers.

Table 4: Decision matrix for microcontroller

		Low Cost	Safety	Response	Simplicity	Rank
Weight		0.1	0.3	0.5	0.1	1
Arduino Uno (9V)	R3	3	6	3	2	3.8
Arduino Uno WIFI (9V)	R4	2	10	10	3	8.5
Arduino Uno (5V)	R3	10	1	1	10	2.8
Arduino Uno WIFI (5V)	R4	5	4	9	8	7
Flight Controller		1	8	10	1	7.6

Starting with cost, an economic setup is always important to consider, especially when working with a budget. Ensuring affordability allows for greater flexibility in allocating funds to other components—particularly since some parts may become faulty or degrade after repeated testing cycles. However, higher cost, higher quality setups often produce better results and reduce the likelihood of issues overall. Due to this tradeoff, and when compared with the other criteria, cost is assigned a relatively lower weight factor.

Safety, as in any engineering application, refers to minimizing harm to both the operator and the system. Ensuring that the drone operates reliably without risks such as electrical damage or malfunctions is essential. It is especially important to design a system capable of handling voltage spikes and high current conditions, which are common when components like motors are pushed beyond their typical operating ranges during difficult tasks. With these considerations in mind, safety is assigned one of the highest weighting factors.

Response is defined as the system’s ability to receive input information, process it, and produce an output over time—in this case, an action. A fast and efficient response is critical for achieving the desired

performance of the design. A controller that meets this requirement enables more effective stabilization and control of the drone. This directly contributes to safe operation, reduced risk of damage from unstable behavior, more accurate PID control, and successful execution of motion commands. These factors are central to the project’s objectives, making response the most heavily weighted criterion in the decision matrix above.

The final criterion is simplicity. Simplicity describes the minimization of complexity of the electronic wiring and control system used in the bicopter drone design. A simpler system is desirable, as it offers several benefits. For example, reduced wiring complexity can improve signal transmission and system responsiveness. It also enhances safety, as fewer wire connections reduce the risk of interference with moving components such as actuators. Additionally, simplicity can decrease the workload required for both hardware assembly and software development. However, many of these challenges can be mitigated through careful design and assembly practices. As a result, simplicity is considered a lower-priority criterion and is assigned a lower weight factor.

While the Arduino UNO R4 (9V) option scored relatively low on the low cost and simplicity categories, it excelled in the safety and response category. Having the highest score in comparison with the others, as it aligned best with the requirements of each respective category. Making it the best choice of microcontroller to control the drone and successfully complete the objectives of this project.

The final design consisted of three mechanical components. The first of the mechanical components was a drone chassis (Figure 2). The top and bottom half of the chassis were 3D printed using PLA and held together by M5 screws. The second mechanical component was the gantry assembly (Figure 3). This consists of the gantry cart and rail setup previously mentioned, as well as supports for the linkage. The linkage is the final mechanical component, comprised of laser-cut acrylic linkages (Figure 4). A schematic of the components joined together is given in Figure 5.

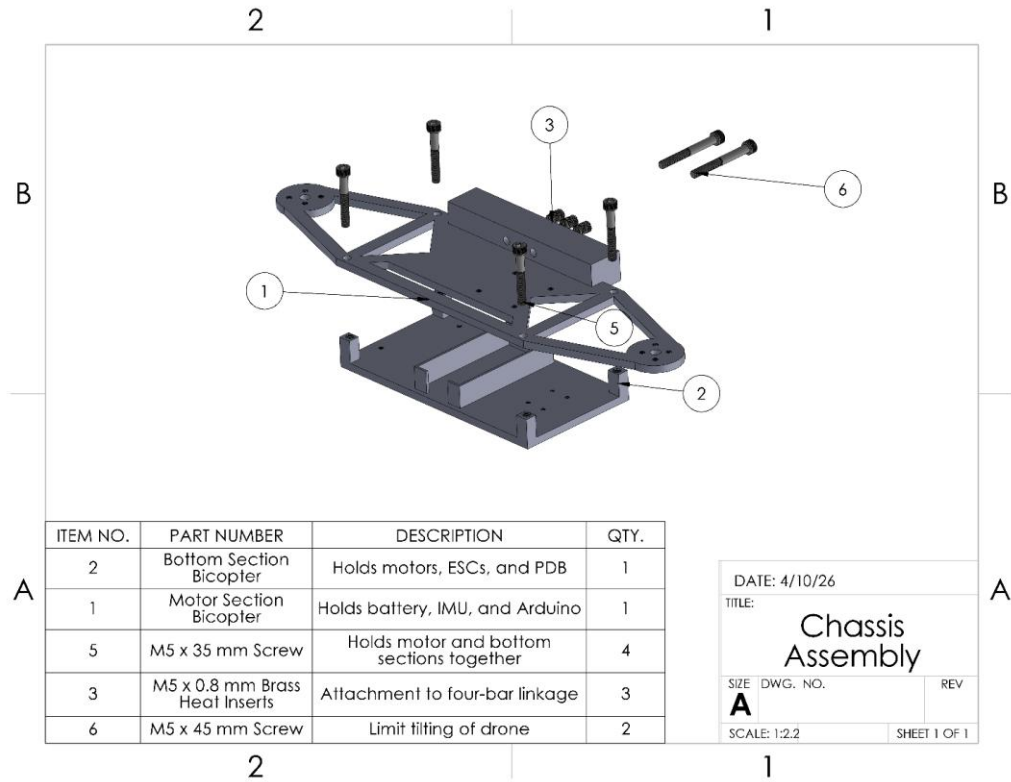


Figure 2: CAD model of Chassis Assembly

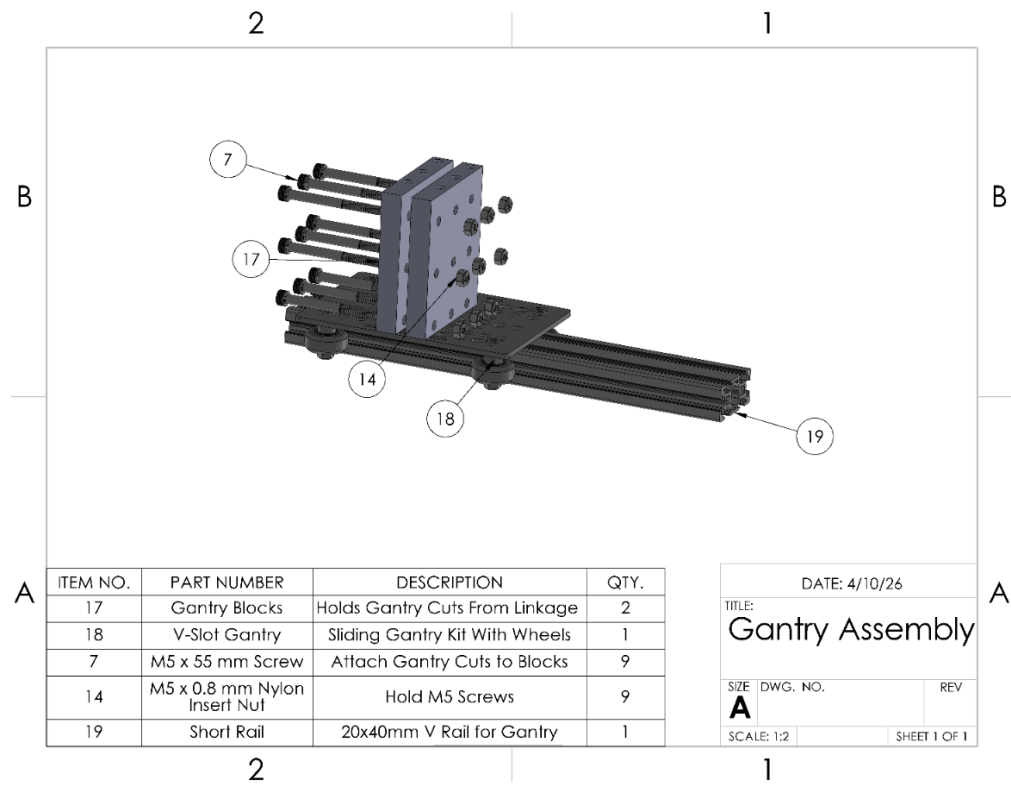


Figure 3: CAD model of Gantry Assembly

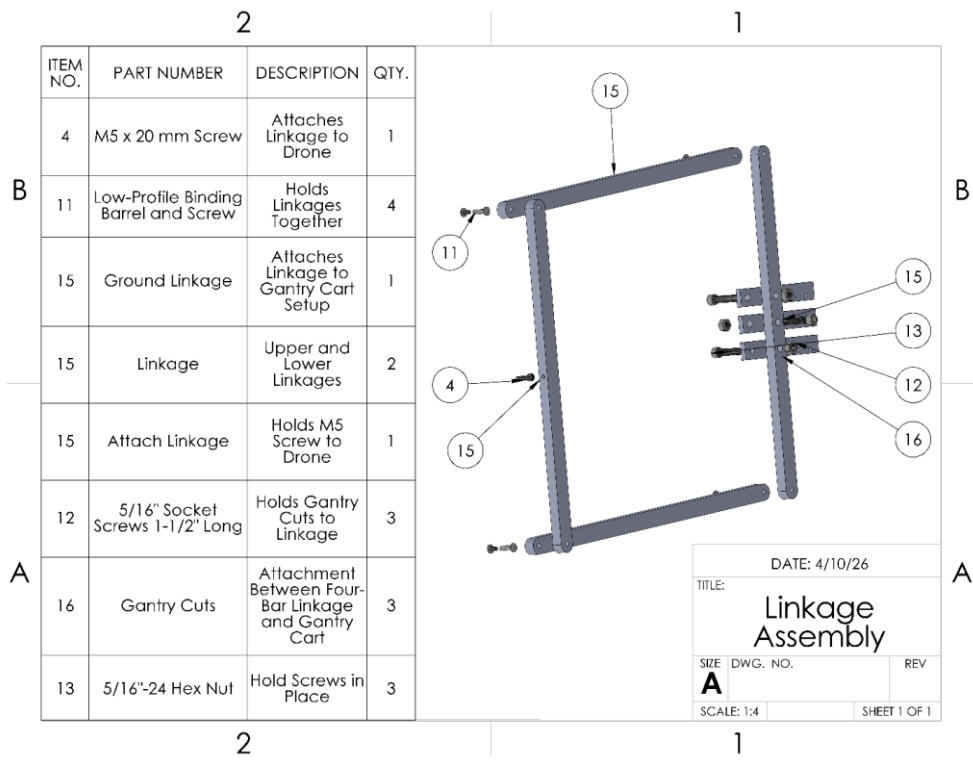


Figure 4: CAD model of Linkage Assembly

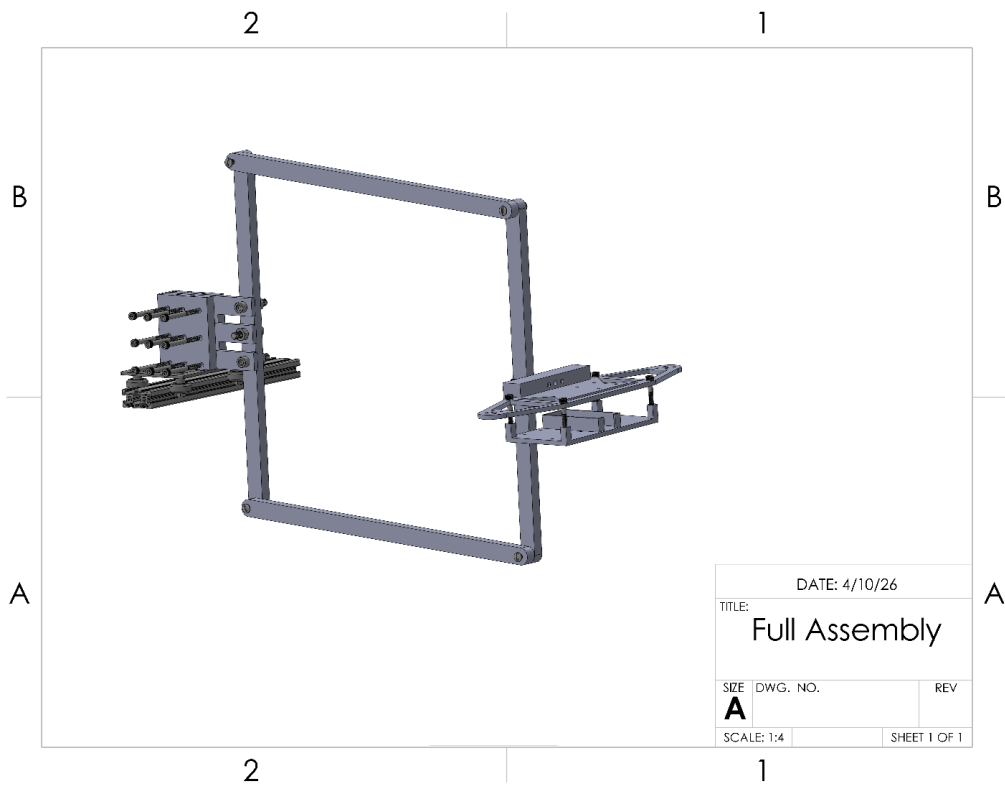


Figure 5: CAD model of complete assembly

The complete set of electronics and necessary wiring is diagramed in Figure 6. The motors had a max current draw of 18A, so 20A ESCs were chosen for safety [21]. The motors were compatible with 3-4S Lipo batteries, so a 4S was chosen in anticipation of high thrust [22]. A PDB [23], with both 12 and 5 volt pads was necessary for powering the ESC's and Arduino (respectively). The MPU5060 IMU was chosen as it is a standard IMU with many Arduino libraries [24]. An Arduino expansion module was used to add more ground ports, and an Arduino joystick was used for a kill switch. It should be noted that the two motors must spin in opposite directions. In general, the three receiver wires on a motor can be attached to any

three of the transmitter pads on an ESC. However, the spin direction will vary. For this device, one motor was soldered to an ESC with an arbitrary orientation. To change the direction of the other motor, two of its receiver wires were soldered to opposite pads on the ESC with respect to the first motor's wire orientation.

A complete bill of materials is given in Table 5. The prices listed are the price of the order, not the unit price. This would mean the price of the full set for fasteners, or the unit price multiplied by the quantity for electronics. Components without a listed price were provided by UIC's Makerspace.

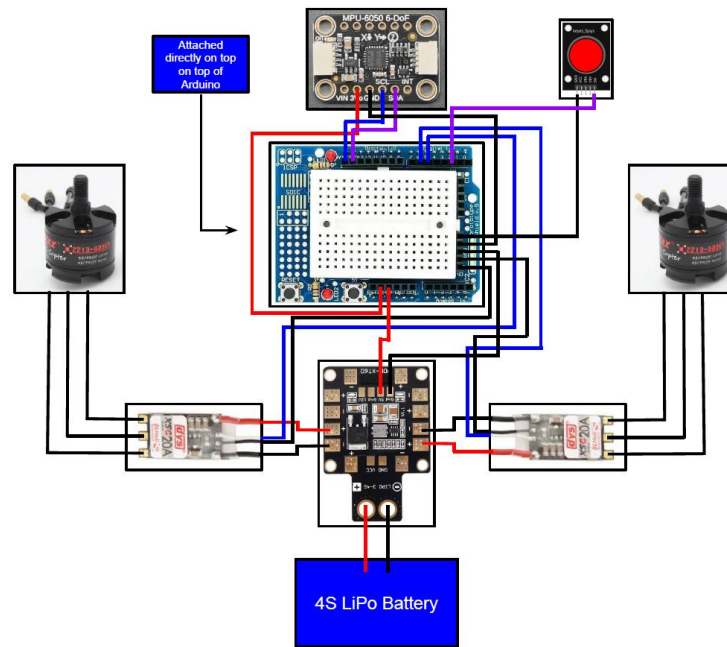


Figure 6: Complete wiring diagram

IV. TESTING

Before combining all the electronics, components were tested individually. First, the IMU was tested with three objectives: to learn the calibration protocol, to ensure angle and acceleration readings were in standard units, and to determine which angle reading would be required for a PD controller considering the IMU's placement on the chassis. Using the "MPU6050_light.h" [27] library, readings for X, Y, and Z angles and accelerations were read and printed (Appendix 2). The y-angle reading the library corresponded with the roll angle of the drone. A small calibration issue was found. The first reading gives an

angle of 20 degrees, descending to 0 within 200 μ s. This was noted and bypassed in the control codes. Next, effectiveness of the PDB was assessed. The XT60 connector was soldered to the PDB and plugged into the battery. A multimeter was used to ensure voltage at all power pads on the PDB. Voltage readings came within 0.03 V of the output specified on the board. Finally, compatibility between the motors and ESCs was confirmed. The motors were soldered to the ESC transmitters, the ESCs' power inputs were attached to a DC power supply with gator clips, and PWM throttles (1-2 microseconds (μ s) range) were sent using the Arduino Servo library. The

compatibility test was successful, so the ESCs' power inputs were soldered to the PDB. Since all electronics tests were successful, the final solder, power from the PDB to the Arduino, was made.

With all the electronics combined, tests were performed with the drone on the linkage. The first goal was to determine the pulse needed for the drone to hover in place. This was determined through trial and error. A pulse was sent to the drone for 10 seconds. One team member would bring the drone up to center position and let go. $1550\text{ }\mu\text{s}$ was the hover speed on the first day of testing. However, after returning to the device the following day, this pulse was too strong. It was found that the ESCs required recalibration often; the ESCs emit a unique audio signal when recalibration is necessary. The calibration code is given in Appendix 3.

Stabilization controls were then implemented. The PD controls for roll stabilization were tested using the code in Appendix 4. Additionally, the logic for the PD controller is diagramed in Figure 7.

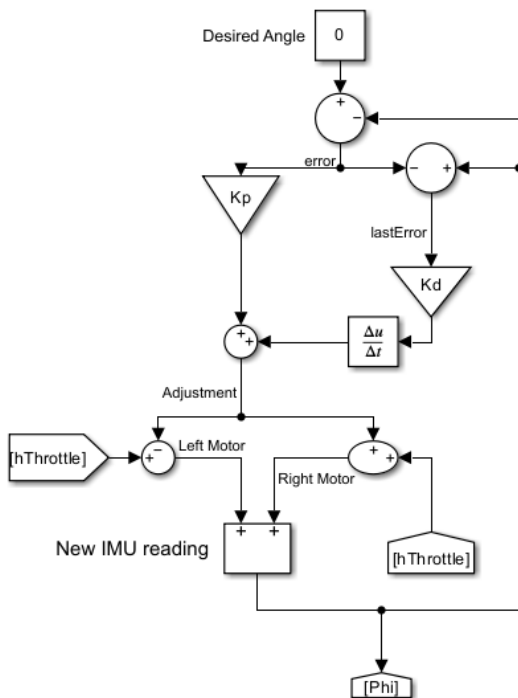


Figure 7: Control logic of the PD controller for roll stabilization

First, the code was tested without the propellers attached to validate the signs of the controller and motors. In other words, when tilted, the lower motor would spin faster than the higher motor. The drone was then placed on the linkage for gain testing. On the first test, chassis tilted too far, causing the propeller to hit the linkage. This was when the extra chassis pins were added to the design. The chassis rotation is physically restricted so the propellers would not hit the linkage. However, for implementation of this design in a lab, where a full range of motion is desired, a wider chassis can be used. This design's chassis was not remodeled to be wider due to UIC Makerspace's printer size constraints.

With the proper control setup, determining the gains was the final step in completing the controls. Coarse tuning was done through visual observation. (Appendix 5). This method was less effective than expected. The team looked into the code and found sign errors. Originally, the PD equation was written as follows in Listing 1.

Listing 1: Original PD transfer function in C++ Arduino code

```
error = setpoint - phi;

if (error > tol) {
    rateError = (error - lastError) / dt;
    float adjustment = (Kp * error) + (Kd * rateError);

    lastError = error;

    pwmRight = hThrottle - adjustment;
    pwmLeft = hThrottle + adjustment;
```

However, the sign of the rate error would cause the wrong motor to speed up during correct. A positive rateError would flip the sign of the adjustment. To remedy this, a forced sign assignment was added to the code (Listing 2).

Listing 2: Sign-adjusted PD transfer function in C++ Arduino code

```

if(phi > 0) {
    sign = 1;
} else if(phi < 0) {
    sign = -1;
} else {
    sign = 0;
}

error = abs(setpoint - phi);

if(abs(error) > tol) {
    rateError = abs((error - lastError)/dt);
    float adjustment = (Kp * error) + (Kd * rateError);
    lastError = error;

    pwmRight = hThrottle + sign * adjustment;
    pwmLeft = hThrottle - sign * adjustment;

```

This adjustment finalized the controller logic. Using the new logic, gain tuning by visual and plot inspection was conducted. The IMU angle, PD adjustment, and both motor throttles were printed to the Arduino serial monitor during testing. These results were converted to CSV files and plotted (Appendix 6). The gains were tuned to mitigate any skips, jittering, overshooting, or slow responses.

V. VALIDATION AND RESULTS

The wiring in Figure 6 was implemented on a PLA printed chassis from Figure 2 (Figures 8-9).

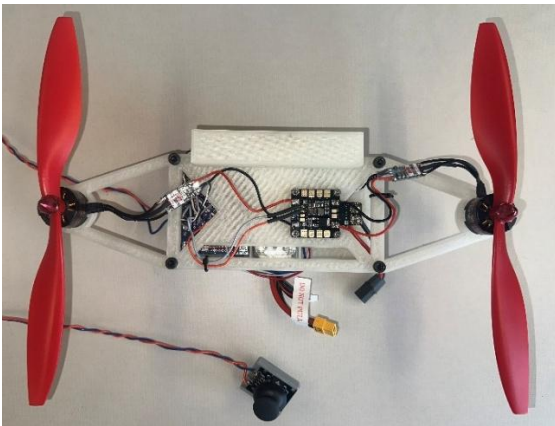


Figure 8: Electronic configuration on top of the chassis.

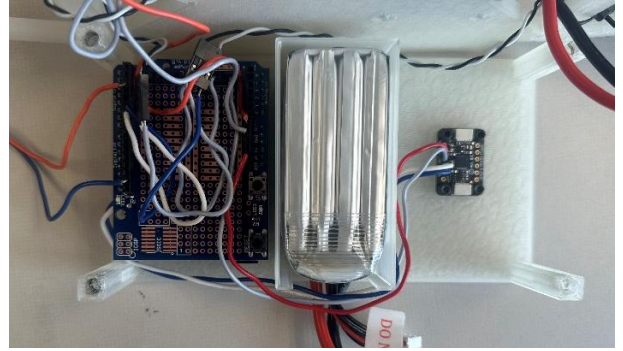


Figure 9: Electronic configuration on the bottom of the chassis

The chassis, linkage, and gantry assembly were attached as designed in Figure 5 (shown in Figure 10).



Figure 10: Complete Assembly

The final PD gains were determined to be: $K_p = 1.5$ and $K_d = 0.03$. These gains corresponded with a quick but safe response. Higher K_p causes overshooting, while a lower K_p slows the response of the motors. A slow response results in a prolonged tilt, causing unwanted horizontal translation. A higher K_d causes jittery motors, while a low K_d also causes slow response.

VI. CONCLUSION

In any sort of engineering application, devices and prototypes are often first fabricated to provide data and simulation for the intention of enhancing said

apparatus to later then fulfil a greater intended purpose. This is a key step in the process of creative thinking and figuring out solutions for a multitude of problems. Testing and compiling multiple iterations of data, then improving based on the data and simulation, are a necessity when it comes to ensuring that implementation of commands and/or additional elements into a final product won't result in failure and unsafe behavior. Instead, have it complete its task to the best of its ability.

This project in particular asks for specific executions with certain constraints that help with this notion of simulating how a bicopter drone would behave, how to optimize its control logic, and finally in turn apply to out-of-the-box operations. As previously mentioned, Professor Bhounsule asked for the construction of a bicopter drone device that will be used for visual demonstration and control theory testing. All for it to be shown to a wide variety of spectators and different environments. Specific constraints within this apparatus include: a dual-motor propeller system to generate drone motion, tethering the bicopter to a surface, restriction to 3 degrees of freedom (y-axis, z-axis, and roll), have an auto stabilization scheme within its roll motion, and finally have restricted motion to a 2D y-z plane. Note that greater emphasis is put on the first three ideas and not so much on that 2D plane motion.

With this said, during this process an abundance of deductions were taken note of. Starting with the main control system, especially for auto stabilization, the IMU. During operation, the IMU would provide data such as the drone's tilt angle to allow the drone to stabilize using a PD controller. However, it was seen that the data compiled by the component would spike or in other words provide "noisy" data ever so often. This data would affect the performance of the drone heavily as motion accuracy depends on it. The implementation of soft layers around the IMU to reduce vibration and filtering the software system through code and additional internal components would help with improving this.

An important function of the project was to be a great visual for students interested and learning concepts such as mechatronics and control systems. Having an observable example of something that

students would only learn theory about would help a lot in gaining a greater understanding, and this project does exactly that. Not only is it cheaper compared to other projects, but it's also not complex in the sense of its fabrication and the equipment used. Making it a great tool and interface sufficient for a classroom and laboratory environment.

As previously mentioned, a prototype for a greater purpose is crucial to successfully determine the most optimal and effective solution to a problem. The device composed in this project does this by setting a great foundation for developing controls in robotics. Permitting others to have a base understanding and explanation when first tackling this problem, allows for further later improvements in a much simpler and more effective process. Presenting the controls and information developed by the team encompasses this idea to its fullest potential, not only with drones but in all sorts of robotics applications.

To further develop this concept of providing a foundation, some potential future work was also considered during the present development stages. These considerations can possibly be incorporated into the system and potentially improve motion and/or controls significantly. The concept of not only having the drone be autonomous but also be combined with manual user control through joysticks on a transmitter has the potential of opening a whole other set of ideas to this problem. Impact forces on each side of the drone were carried out to prove its auto stabilization feature. However, placing the drone in harsh environments and testing controls there can be major when thinking about prototyping for package delivery drones for example. These applications deal with a variety of forces such as wind; something more continuous and not just a sudden impulse. To go along with this, working on autonomous path following to observe movement accuracy with path complexity can also be a vital testing subject when it comes to outside applications like the one mentioned above.

Overall, these were all ideas obtained and taken note of during the entirety of the progress and later finalization of the tethered bicopter motor stabilization study. Achieving the sponsor's requests from its assembly to its examinations to what can be done in the future.

VII. APPENDIX

1. Complete bill of materials for the build

Part Number	Part Name	Quantity	Material	Cost
1	Motor Section Bicopter	1	PLA	-
2	Bottom Section Bicopter	1	PLA	-
3	M5 x 0.8mm Heat Inserts	3	Brass	-
4	M5 x 20 mm Screw	1	Black-Oxide Alloy Steel	-
5	M5 x 35 mm Screw	4	Black-Oxide Alloy Steel	-
6	M5 x 45 mm Screw	2	Black-Oxide Alloy Steel	-
7	M5 x 55 mm Screw	9	Black-Oxide Alloy Steel	-
8	M3 x 5 mm Screw	8	Zinc-Plated Alloy Steel	-
9	M3 x 8 mm Screw	8	Zinc-Plated Alloy Steel	-
10	M2.2 x 3mm Screw	4	Black-Oxide Alloy Steel	-
11	Low-Profile Binding Barrel and Screw [25]	4	Aluminum	\$7.58
12	5/16" Socket Screw 1-1/2" Length	3	Brass	-
13	5/16"-24 Hex Nut	3	Zinc-Plated Steel	-
14	M5 x 0.8 mm Insert Nut	9	Nylon	-
15	15.75 Inch Linkages	4	Acrylic	-
16	Gantry Cuts	3	Acrylic	-
17	Gantry Blocks	2	PLA	-
18	Gantry Bundle (Gantry Plate and Wheels)	1	Steel	\$34.19
19	20x40x1000 mm V-Rail	1	Aluminum	\$13.25
20	Clamp Supports	2	Acrylic	-
21	C-Clamps	2	-	-
22	EMAX MT Series 935kv Brushless Motor With 1045 Prop	2	-	\$31.00
23	DYS XSC-20A BLHeli_S ESC	2	-	\$33.98
24	Adafruit MPU 6050	1	-	\$12.95
25	Tattu FunFly 1550 mAh 4s 100 C Lipo Battery	2	-	\$56.98
26	Arduino Uno R4	1	-	-
27	Arduino Prototype Expansion Module	1	-	-

Part Number	Part Name	Quantity	Material	Cost
28	Arduino Joystick	1	-	-
29	Speedy FPV Power Distribution Board XT60	1	-	\$5.65
30	22 Gauge Wires	4	Copper	-
31	20 Gauge Wires	14	Copper	-
32	XT60H 12/14AWG Male Pigtail [26]	1	-	\$9.99
			TOTAL:	\$205.57

2. C++ Arduino code with angle and acceleration readings using the MPU6050_light.h library [27].

```

#include <MPU6050_light.h>
#include <Wire.h>

MPU6050 mpu(Wire);

void setup(void) {

  Serial.begin(115200);
  Wire.begin();

  // Initialize IMU
  byte status = mpu.begin();
  Serial.print(F("MPU6050 status: "));
  Serial.println(status);
  while(status != 0){ } // Stop if IMU not found

  Serial.println(F("Calculating offsets"));
  mpu.calcOffsets();
  delay(6000);
}

void loop() {

  mpu.update();

  float xAngle = mpu.getAngleX();
  float yAngle = mpu.getAngleY();
  float zAngle = mpu.getAngleZ();

  Serial.print("X-Angle: "); Serial.print(xAngle); Serial.print(" ");
  Serial.print("Y-Angle: "); Serial.print(yAngle); Serial.print(" ");
  Serial.print("Z-Angle: "); Serial.print(zAngle); Serial.print(" ");
  Serial.print("\n");

  delay (10);
}

```

3. C++ Arduino code for calibrating ESCs [28]

```
//ESC calibration sketch; author: ELECTRONOOBS

#include <Servo.h>
int MAX_SIGNAL=2000;
int MIN_SIGNAL = 1000;
int rMOTOR_PIN = 5;
int lMOTOR_PIN = 6;
int DELAY = 1000;

Servo rmotor;
Servo lmotor;

void setup() {
  Serial.begin(9600);
  Serial.println("ELECTRONOOBS ESC calibration...");
  Serial.println(" "); delay(10000);
  Serial.println("Program begin...");
  delay(1000);
  Serial.println("This program will start the ESC.");
  rmotor.attach(rMOTOR_PIN);
  lmotor.attach(lMOTOR_PIN);
  Serial.print("Now writing maximum output: (");
  Serial.print(MAX_SIGNAL);
  Serial.print(" us in this case)");
  Serial.print("\n");
  Serial.println("Turn on power source, then wait 2 seconds and press any key.");
  rmotor.writeMicroseconds(MAX_SIGNAL);
  lmotor.writeMicroseconds(MAX_SIGNAL);
  // Wait for input
  while (!Serial.available());
  Serial.read();
  // Send min output
  Serial.println("\n"); Serial.println("\n");
  Serial.print("Sending minimum output: (");
  Serial.print(MIN_SIGNAL);
  Serial.print(" us in this case)");
  Serial.print("\n");
  rmotor.writeMicroseconds(MIN_SIGNAL);
  lmotor.writeMicroseconds(MIN_SIGNAL);
  Serial.println("The ESC is calibrated");
  Serial.println("----");
  Serial.println("Now, type a values between 1000 and 2000 and press enter");
```

```

Serial.println("and the motor will start rotating.");
Serial.println("Send 1000 to stop the motor and 2000 for full throttle");
}
void loop() { if (Serial.available() > 0) {
    int DELAY = Serial.parseInt();
    if (DELAY > 999) {
        rmotor.writeMicroseconds(DELAY);
        lmotor.writeMicroseconds(DELAY);
        float SPEED = (DELAY-1000)/10;
        Serial.print("\n");
        Serial.println("Motor speed:");
        Serial.print(" ");
        Serial.print(SPEED);
        Serial.print("%"); }
    }
}

```

4. C++ Arduino code with PD controls for roll stabilization.

```

#include <Servo.h>
#include <MPU6050_light.h>
#include <Wire.h>

MPU6050 mpu(Wire);

int killpin = 2;
volatile bool killed = false;

const int rESCpin = 5;
const int lESCpin = 6;

Servo rESC;
Servo lESC;

const int initPWM = 1000;
const int minPWM = 1050;
const int maxPWM = 1650;

int hThrottle = 1550;

int tol = 0.5;
double Kp = 1.5;
double Kd = 0.03;
double setpoint = 0;

```

```

float error, lastError, rateError;
double dt = 0.01;
int sign = 0;

int pwmRight = hThrottle;
int pwmLeft = hThrottle;

void handleKill() {
    killed = true;
    rESC.writeMicroseconds(0);
    lESC.writeMicroseconds(0);
}

void setup(void) {
    Serial.begin(115200);
    delay(1500);

    Wire.begin();

    pinMode(killpin, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(killpin), handleKill, FALLING);

    lESC.attach(lESCpin);
    rESC.attach(rESCpin);

    byte status = mpu.begin();
    Serial.print(F("MPU6050 status: "));
    Serial.println(status);
    while(status != 0){ }

    Serial.println("Calculating offsets, do not move IMU...");
    mpu.calcOffsets();

    Serial.println("Arming ESCs...");
    rESC.writeMicroseconds(initPWM);
    lESC.writeMicroseconds(initPWM);
    delay(10000);
    Serial.println("ESCs Armed.");

    rESC.writeMicroseconds(hThrottle);
    lESC.writeMicroseconds(hThrottle);
    delay(5000);

    for (int i = 0; i < 200; i++) {
        mpu.update();
    }
}

```

```

    delay(1);
}

Serial.println("IMU Calibrated.");
}

void loop() {
    if (killed) {
        rESC.writeMicroseconds(0);
        lESC.writeMicroseconds(0);
        while(1);
    }

    mpu.update();
    float phi = mpu.getAngleY();

    if (phi > 0) {
        sign = 1;
    } else if (phi < 0) {
        sign = -1;
    } else {
        sign = 0;
    }

    error = abs(setpoint - phi);

    if (abs(error) > tol) {
        rateError = abs((error - lastError)/dt);
        float adjustment = (Kp * error) + (Kd * rateError);

        lastError = error;

        pwmRight = hThrottle + sign * adjustment;
        pwmLeft = hThrottle - sign * adjustment;

        pwmRight = constrain(pwmRight, minPWM, maxPWM);
        pwmLeft = constrain(pwmLeft, minPWM, maxPWM);

        rESC.writeMicroseconds(pwmRight);
        lESC.writeMicroseconds(pwmLeft);

        Serial.print(phi);
        Serial.print(",");
        Serial.print(adjustment);
        Serial.print(",");
    }
}

```

```

    Serial.print(pwmRight);
    Serial.print(",");
    Serial.println(pwmLeft);
}

else {
    float adjustment = 0;

    pwmRight = hThrottle;
    pwmLeft = hThrottle;

    rESC.writeMicroseconds(pwmRight);
    lESC.writeMicroseconds(pwmLeft);

    Serial.print(phi);
    Serial.print(",");
    Serial.print(adjustment);
    Serial.print(",");
    Serial.print(pwmRight);
    Serial.print(",");
    Serial.println(pwmLeft);
}
delay(10);
}

```

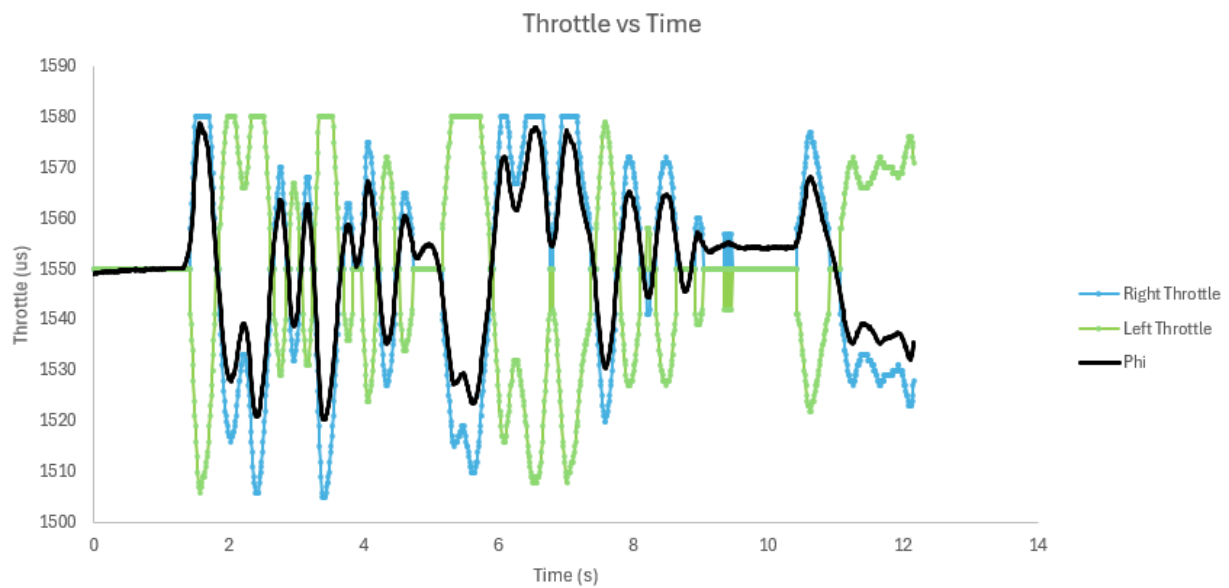
5. Coarse tuning results by visual observation (4/16/26-4/20/26)

Throttle	Kp	Kd	Tolerance	Notes
1550	2	1.6	5	Getting stuck
1550	2	1.6	3	Getting stuck
1560	2.5	1.6	3	
1558	2.5	1.6	3	
1561	2.5	2	3	
1558	2.5	2	3	This was run with a fully charged battery & 1558 is too strong
1540	2.5	2	3	
1540	2.5	2	1.5	
1530	2.8	2.8	1.5	
1530	2.5	2	1	Low tolerance is better, slow to adjust

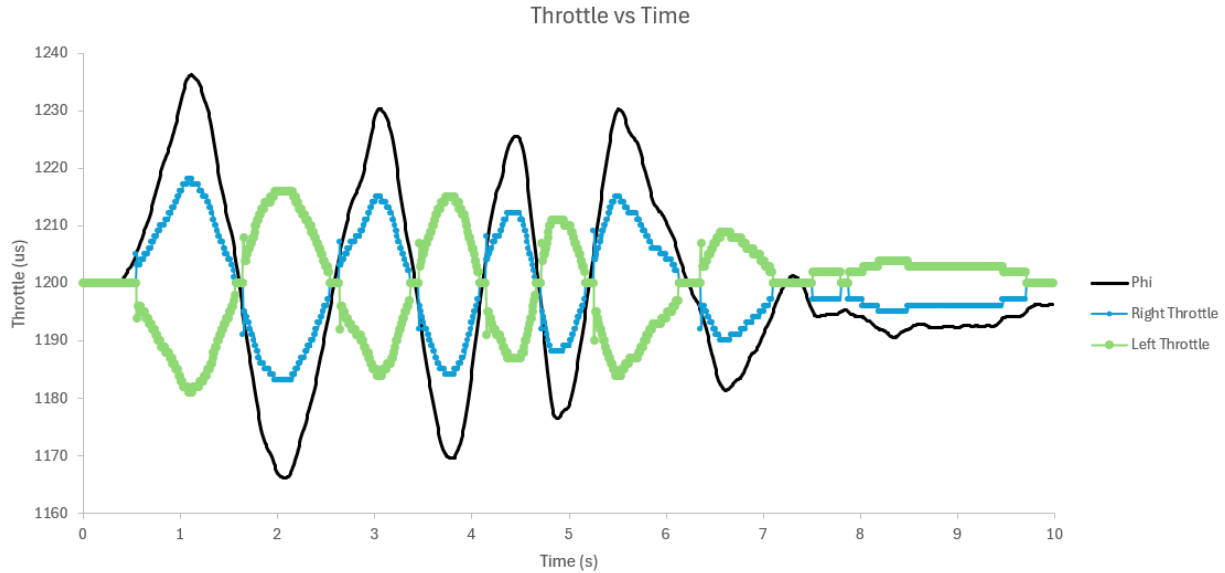
Throttle	Kp	Kd	Tolerance	Notes
1530	2.7	2	1	Too jumpy
1555	2.5	2.1	1	Not working
1555	1	0.8	1	Not leveling
1530	2.5	2	1	Too fast (new code may not have uploaded correctly)
1530	2	2	1	
1530	3	1.5	1	Numbers look good, less jumpy
1530	1.2	0.9	1	Ran well w/o the propellers
1530	1.5	0.5	1	
1530	1.2	0.9	1	
1270	1	0.05	1	Not leveling
1270	1.5	0.05	1	No output on the serial monitor
1270	2	0.05	1	Not leveling- increase the Kp and Kd (Right motor was reaching max and not adjusting)
1270	2	0.08	1	Adjustment not working- flip the pins
1270	2	0.08	1	Flipped the pins in the code and still no difference

6. Plots from tests for gain tuning

a. Throttle vs time for $K_p = 1.5$, $K_d = 0.5$



b. Throttle vs time for $K_p = 0.5$, $K_d = 0.5$



VIII. CITATIONS

- [1] J. Krause, A. Alaeddini, and P. A. Bhounsule, "Gaussian process regression for SIM-to-real transfer of hopping gaits," *Volume 8: 47th Mechanisms and Robotics Conference (MR)*, Aug. 2023. doi:10.1115/detc2023-116959
- [2] S. Zhao and J. Yu, "Three-dimensional obstacle avoidance system for indoor quadrotor drones," China Patent CN105629985B, 4 Sep. 2018.
- [3] C.-M. Yang and P. A. Bhounsule, "Koopman operator based linear model predictive control for 2D quadruped trotting, bounding, and gait transition," *2025 American Control Conference (ACC)*, pp. 1089–1094, Jul. 2025. doi:10.23919/acc63710.2025.11107511
- [4] J. Iglesias and M. Aguinaga, "Aerial vehicles with uncoupled degrees of freedom," U.S. Patent 12 037 109 B2, 16 Jul. 2024.
- [5] Q. Zhang, Z. Liu, J. Zhao and S. Zhang, "Modeling and attitude control of Bi-copter," 2016 IEEE International Conference on Aircraft Utility Systems (AUS), Beijing, China, 2016, pp. 172-176, doi: 10.1109/AUS.2016.7748042.
- [6] Federal Aviation Administration, "Drone Integration Concept of Operations (May 2025)," 2025. [Online]. Available: <https://www.faa.gov/uas/resources/policylibrary/Drone-Integration-Concept-of-Operations-May-2025.pdf>.
- [7] Federal Aviation Administration, "14 CFR Part 107 -- Small Unmanned Aircraft Systems," *www.ecfr.gov*, Jun. 28, 2016.
- [8] "Checklist for Tethered Public Safety 49 USC §44806 Operations." Available: https://www.faa.gov/uas/public_safety_gov/public_safety_toolkit/Checklist-for-Public-Safety-Use-of-Actively-Tethered-UAS-49-USC-44806-August-2024.pdf
- [9] "Occupational Safety and Health Administration," Dec. 2021. Available: https://www.osha.gov/sites/default/files/enforcement/directives/CPL_02-01-169.pdf
- [10] "V-Slot 20 x 40mm - Maker Store USA," *Maker Store USA*, Apr. 2026. https://makerstore.cc/product/v-slot-20-x-20mm/?attribute_length=1000mm
- [11] "V-Slot Gantry Bundle - Maker Store USA," *Maker Store USA*, Oct. 08, 2025. <https://makerstore.cc/product/v-slot-gantry-bundle-2/>

- [12] “5 foot lightweight camera jib with bag set kit,” CobraCrane, https://cobracrane.com/products/bpulb?variant=17530900167&country=US¤cy=USD&utm_medium=product_sync&utm_source=google&utm_content=sag_organic&utm_campaign=sag_organic&srsltid=AfmBOorZsPRBbJBxycRGCwuIxsXeKf7NIJnPmLqgw8K700Kzr_p1GjnRe7Q
- [13] Faye.z S. Ahmed, Salah Laghrouche, Mohammed El Bagdouri, Analysis, modeling, identification and control of pancake DC torque motors: Application to automobile air path actuators, *Mechatronics*, Volume 22, Issue 2, 2012, Pages 195-212, ISSN 0957-4158
- [14] SunnySky USA, “SunnySky X2212 KV980 Brushless Motors – Multirotor Version No Accessory,” Sunnyskyusa.com. [Online]. Available: <https://sunnyskyusa.com/products/sunnysky-x2212-brushless-motors-new?srsltid=AfmBOooHBB3RXkh4nCoO6Ju1axFGMwnV5kyolkrL98g60YKcbGDosdGH>.
- [15] SpeedyFPV, “4pcs 5010 Brushless Motors 750 KV 2-6S High Torque Motor Set for 14”–16” Propellers,” SpeedyFPV. [Online]. Available: <https://speedyfpv.com/products/4pcs-5010-brushless-motors-750kv-2s-6s-high-torque-motor-set-for-14-16-propellers>
- [16] EMAX USA, “MT Series - 2213 935kv Brushless Motor With 1045 Prop,” *Emax USA*. [Online]. Available: <https://emax-usa.com/collections/22xx/products/emx-mt-1534-emax-multicopter-motor-mt2213-with-prop1045-combo-935kv>. [Accessed: Apr. 11, 2026].
- [17] AliExpress, “Product 2251832291755325 – Specifications,” AliExpress. [Online]. Available: <https://www.aliexpress.us/item/2251832291755325.html?gatewayAdapt=glo2usa4itemAdapt#nav-specification>.
- [18] Amazon, “FLASH HOBBY Brushless Multirotor Multicopter Motor,” Amazon.com. [Online]. Available: <https://www.amazon.com/FLASH-HOBBY-brushless-multirotor-multicopter/dp/B08LN1MJ7X>.
- [19] Deepak BBVL, Davinder Pal Singh, Sampath Kumar Kuppa, M Jayanthi Rao, Design optimization of drone BLDC motor for delivery service applications, *Materials Today: Proceedings*, 2023, ISSN 2214-7853
- [20] M. S. Islam, I. Husain and R. Mikail, “Slotless lightweight motor for drone applications,” *2017 IEEE Energy Conversion Congress and Exposition (ECCE)*, Cincinnati, OH, 2017, pp. 5041-5048, doi: 10.1109/ECCE.2017.8096851
- [21] “DYS XSC-20A BLHeli_S ESC,” www.getfpv.com, 2026. <https://www.getfpv.com/dys-xsc-20a-blheli-s-esc.html>.
- [22] “Tattu FunFly 1550mAh 4s 100C Lipo Battery,” www.getfpv.com, 2026. <https://www.getfpv.com/batteries/mini-quad-batteries/tattu-funfly-1550mah-4s-100c-lipo-battery.html>
- [23] “Drone Power Distribution Board XT60 3-4S 9-18V 5V 12V Output PDB | SpeedyFPV,” Speedyfpv.com, 2022. https://speedyfpv.com/products/drone-power-distribution-board-xt60-3-4s-9-18v-5v-12v-output-pdb?utm_source=shop_app&list_generator=link_to_storefront&context=product&user_id=287522598&variant=8596736049203
- [24] A. Industries, “Adafruit MPU-6050 6-DoF Accel and Gyro Sensor - STEMMA QT Qwiic,” www.adafruit.com. <https://www.adafruit.com/product/3886>
- [25] “McMaster-Carr,” Mcmaster.com, 2025. <https://www.mcmaster.com/99024A366/>
- [26] “XT60H 12/14AWG male/female pigtail - black (3pcs),” www.getfpv.com, <https://www.getfpv.com/xt60h-12-14awg-male-female-pigtail-black.html>
- [27] Rfetic, “MPU6050_light”, GitHub. https://github.com/rfetic/MPU6050_light, January 2021.
- [28] Electronoobs. “ESC Calibrate with Arduino Code.” *Electronoobs*, 3 Apr. 2017, electronoobs.com/eng_robotica_tut5_1_code1.php. Accessed 30 Apr. 2026.